Multi-LCB: Extending LiveCodeBench to Multiple Programming Languages

Maria Ivanova [2,*] Pavel Zadorozhny [1,*] Rodion Levichev [1,*] Ivan Petrov [1] Pavel Adamenko [1] Ivan Lopatin [1] Alexey Kutalev [1] Dmitrii Babaev [1]

[1] GigaCode [2] Yandex School of Data Analysis, Applied AI Institute * Equal contribution

https://github.com/Multi-LCB/Multi-LCB

Problem statement

- LiveCodeBench (LCB) is a framework of continuously curating competitive-programming problems, filtering them by release date, and enabling contamination-aware evaluation. As a result, LCB has quickly become a standard benchmark for evaluating LLMs on **Python** code generation.
- Prior work extend code generation evaluation beyond Python by rewriting function signatures and regenerating unit tests for each language. Even a simple Python assertion like:

```
assert binomial_coeff(5, 2) == 10
```

must be expanded into multi-line Java test code. This translation must be repeated separately for every language and is sensitive to syntax and runtime differences.

Can LLM generalize across multiple programming languages?

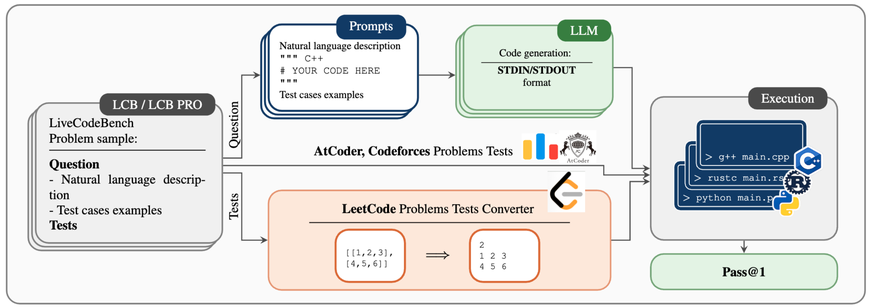

Figure 1. **Top**: LCB/LCB-Pro natural-language problem descriptions are wrapped into prompts specifying the target programming language and passed to the LLM for STDIN/STDOUT code generation. AtCoder and Codeforces problem tests are passed directly to the execution stage. **Bottom**: LeetCode problem tests are transformed through **test converter** to produce equivalent STDIN/STDOUT inputs. The generated code is compiled or executed in the target programming language and evaluated (Pass@1).

Key Findings

- *Python is not always a reliable proxy for individual non-Python languages.* Our results reveal substantial and practically meaningful performance gaps across languages.
- *Python overfitting.* Models that perform strongly in Python often degrade sharply in other languages.
- *Language-specific contamination.* Evidence of data leakage varies by programming language, reflecting uneven distribution in pretraining corpora.
- *Substantial multi-programming-language disparities.* Models show large performance gaps across languages, with weaker results in statically typed or less prevalent languages.

Model Performance Results

Table 1. Performance results on Multi-LCB for the tasks from February 2025 till May 2025. Scores represent the **Pass@1 (%)** metric averaged on 10 runs. Higher is better, **bold** is best, *italic* is the second best. Temperature t=0.2 (* - reasoning mode)

Model	Python	C++	Java	Go	JS	TS	C#	Rust	Ruby	PHP	Kotlin	Scala	Avg
Gpt-oss-120B* (Medium)	71.1	72.3	70.4	**69.9**	**70.5**	**70.3**	57.3	**70.5**	**70.2**	*66.1*	**71.0**	54.1	**67.8**
Qwen3-235B-A22B-Thk-2507*	**74.0**	**75.8**	**73.9**	56.7	*67.0*	*62.5*	*66.5*	47.7	49.4	**69.0**	*67.7*	*57.6*	*64.0*
DeepSeek-R1-0528*	66.3	68.0	67.8	55.0	64.6	58.9	61.6	*63.1*	*62.4*	61.6	66.0	**62.3**	63.1
Gpt-oss-20B* (Medium)	63.6	65.7	62.7	*59.9*	61.9	61.8	52.4	61.9	61.7	60.5	62.4	43.1	59.8
Qwen3-30B-A3B-Thk-2507*	64.0	65.7	62.4	44.1	51.9	46.5	56.5	51.7	42.1	58.8	50.6	43.6	53.2
Gpt-oss-120B* (Low)	56.0	55.4	56.8	51.8	55.9	55.6	45.7	56.0	53.0	53.4	55.8	42.2	53.1
Qwen3-235B-A22B*	58.9	58.3	55.0	48.7	50.0	48.8	51.0	40.7	46.6	48.4	47.5	33.6	48.9
Qwen3-32B*	57.6	55.3	56.0	42.1	49.6	49.3	49.1	40.1	52.1	50.0	46.4	35.6	48.6
Qwen3-30B-A3B*	55.0	51.5	50.6	36.9	49.9	48.2	43.9	38.4	46.8	48.0	44.3	32.2	45.5
Gpt-oss-20B* (Low)	46.2	47.9	46.3	42.6	45.1	42.7	41.2	42.0	44.7	45.8	46.3	29.2	43.3
Qwen3-14B*	53.5	47.2	47.2	32.4	45.0	46.0	43.3	31.5	45.3	45.5	39.2	32.4	42.4
Qwen3-235B-A22B-Instr-2507	43.8	42.7	45.5	35.0	26.4	19.5	44.1	39.5	41.5	42.4	41.1	28.1	37.5
Qwen3-8B*	46.3	39.7	36.7	25.8	36.5	38.8	36.3	20.5	39.5	36.0	24.0	27.0	33.9
Qwen3-Coder-30B-A3B-Instr	36.6	31.1	35.3	25.8	28.4	28.0	34.7	34.3	34.7	31.8	35.7	20.2	31.4
Qwen3-30B-A3B-Instr-2507	38.9	35.6	37.2	22.4	20.8	18.2	36.5	32.1	34.7	34.8	35.9	25.7	31.1
Qwen2.5-Coder-32B-Instr	27.5	26.9	30.5	23.9	6.3	28.8	28.5	24.7	24.6	27.3	26.6	24.5	25.0
Seed-Coder-8B-Instr	22.1	23.4	26.0	22.1	23.3	23.1	27.0	21.8	21.6	20.4	23.4	21.8	23.0
OpenRsn-Nmt-32B*	64.4	44.2	40.8	11.5	10.8	10.5	29.9	2.8	18.3	15.8	17.3	6.0	22.7
DeepSeek-R1-Distill-Qwen-32B*	39.4	22.2	33.2	11.9	16.2	11.6	29.3	20.2	40.1	12.9	20.5	7.0	22.0
Devstral-Small-2505*	23.2	22.6	22.8	16.1	22.7	24.7	24.1	19.9	19.9	20.8	21.2	17.2	21.3
Qwen2.5-Coder-14B-Instr	22.0	21.3	23.9	19.2	22.6	17.5	23.3	16.7	22.7	22.7	18.1	20.4	20.9
OpenCodeRsn-Nmt-1.1-32B*	56.0	37.3	33.1	9.9	8.2	4.9	25.5	1.1	23.4	19.3	12.3	7.0	19.8
DeepSeek-R1-Distill-Qwen-14B*	41.8	16.3	24.9	10.8	10.2	11.5	29.2	3.7	34.5	3.8	11.5	3.3	16.8
Deepseek-Coder-33B-Instr	17.2	16.2	18.5	12.4	8.5	7.4	17.1	2.6	15.2	16.5	16.0	12.2	13.3

Our results reveal substantial and practically meaningful **performance gaps across languages**. For example, Gpt-oss-120B* (Medium) outperforms Qwen3-235B-A22B-Thk-2507* on Go, Javascript, Typescript, Rust, Ruby and Kotlin, and DeepSeek-R1-0528* outperforms Qwen3-235B-A22B-Thk-2507* on Rust, Ruby and Scala, despite Qwen3-235B-A22B-Thk-2507* being consistently stronger on Python.

This is precisely why strong Python ability is not always a reliable proxy for true cross-lingual code generation competence and evaluation must consider performance in the target languages rather than relying on Python alone.

Strong Python ability is Not Always a Reliable Proxy

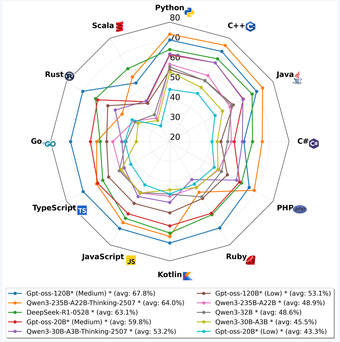

Figure 2. Top-10 models by Pass@1

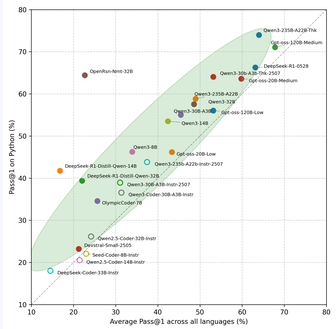

Figure 3. Scatter of Python vs. Average Pass@1

Almost every point lies above the diagonal, demonstrating a consistent **bias toward Python**. OpenRsn-Nmt-32B* show the starkest gap, exceeding 60% on Python while remaining below 30% across other languages. Even the largest reasoning-augmented models retain a measurable positive bias toward Python.